

EE108 Final Project Report

Berwyn Berwyn
Stanford University

Arif Ismail
Stanford University

1 Music Player Implementation

This module handles all the sound generation part of the project. It is designed to be able to execute every mode. There are two different datapaths: one for modes NORMAL, REWIND, and FAST FORWARD; and another one for GENERATE.

For the three modes we used the stored information in the song_rom and was able to generate chords. However, for the GENERATE mode, we feed the nn_song_reader data from the neural network, and only able to play one note at a time.

2 Song Reader Implementation

Song reader reads instructions from song_rom and assigns them to note_players. Note_player _1 is prioritized, then 2, 3, and 4. In the case of mode == 'GENERATE', we can directly assign the instructions to a single note_player.

3 Note Player Implementation

This modules handles playing a single note. It contains 4 sine_readers to create the harmonics. They have weights that are predetermined based on which harmonic the note_player wants to play.

4 Sine Reader Implementation

This module generates a sine wave given the step_size (frequency). It also accounts for reverse sampling when in REWIND and sampling twice as fast when in FAST FORWARD.

5 Enhanced Waveform Implementation (in top module)

This setup of 5 waveform_display_top modules ensures that all 4 chords and the resultant wave is all being displayed. Waves overlap with the resultant wave being prioritized. In GENERATE, only one wave will be displayed, hence the sample output is directly connected to wave_display_top_overall.

6 Neural Network Implementation

This module implements a two layer neural network. The module takes in a 96-bit context vector `ctxt` as input and outputs a 12-bit new note where each bit represents whether a particular output neuron is fired.

Architecture: We have a 96-bit input where each bit go to a layer of 32 hidden neurons (we call this the hidden layer). The output from the hidden layer is then fed to an output layer consisting of 12 neurons. All weights and biases needed for computations are stored in four ROMs (`weight_hidden_rom`, `bias_hidden_rom`, `weight_output_rom`, and `bias_output_rom`) in 16-bit fixed point format and 12 fractional bits (Q4.12).

Computation: The module sequentially computes each neuron multiply and accumulate operation at a time, reusing a single multiplier and accumulator. During the HIDDEN phase, it iterates over all $96 \times 31 = 3072$ cycles, where for each cycle, the dot products of the weights and the weights are computed. Each output from the hidden neuron passes through a ReLU activation function and is stored in one of the 32 registers. During the OUTPUT phase, it iterates over all $32 \times 12 = 384$ cycles, where the dot products against the stored hidden activations are computed.

Control: We have a 4-state FSM (IDLE, HIDDEN, OUTPUT, DONE) represented by 3 bits. We also implemented a 3-stage pipeline (ROM fetch, operand mux, multiply). This results in all accumulation and writeback logic operating on state signals delayed by 3 cycles (`st_s3`, `li_s3`, `lh_s3`, etc).

Output: The new note output is valid when `nn_done` pulses high for exactly one cycle after the state DONE is reached.

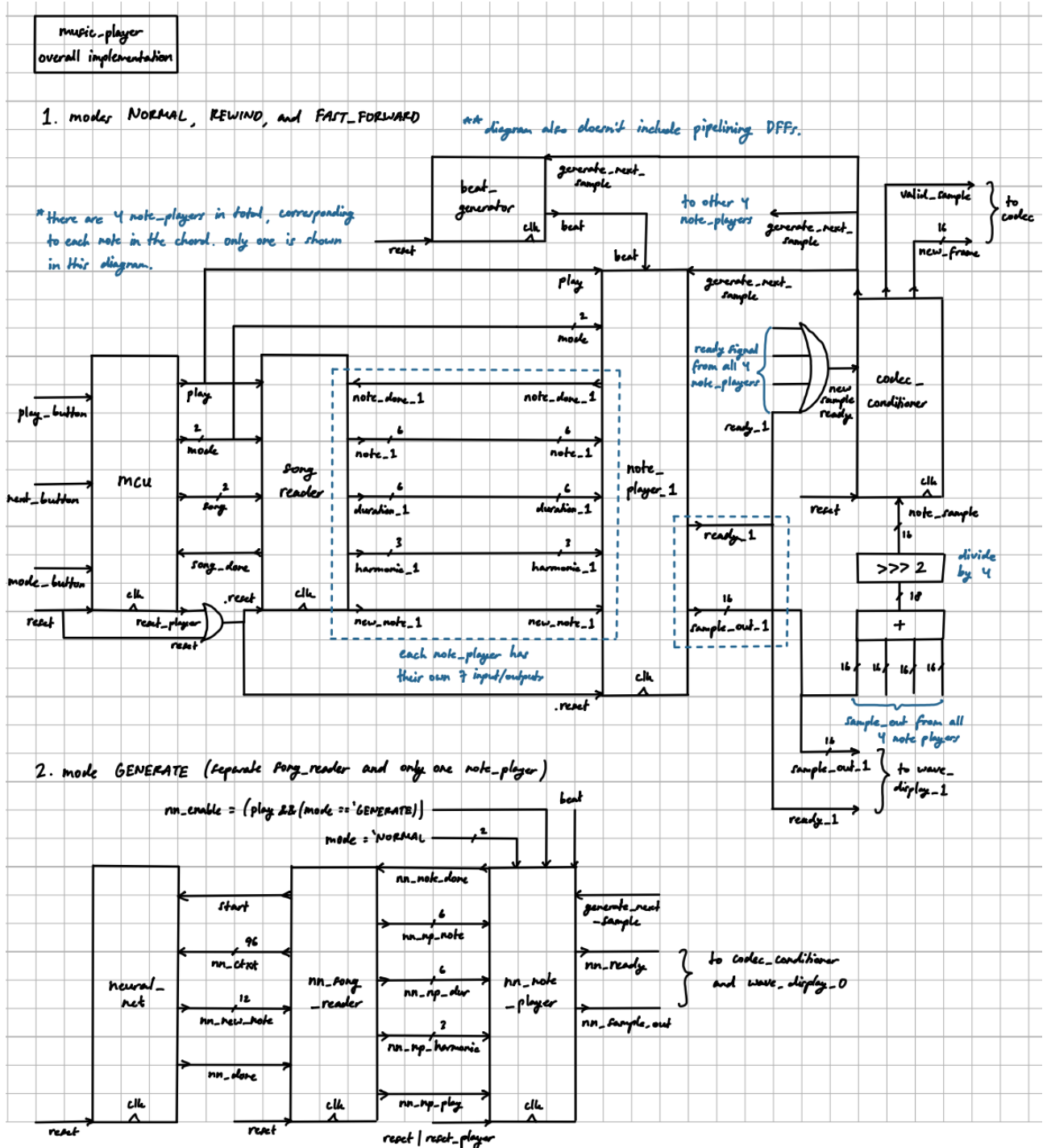
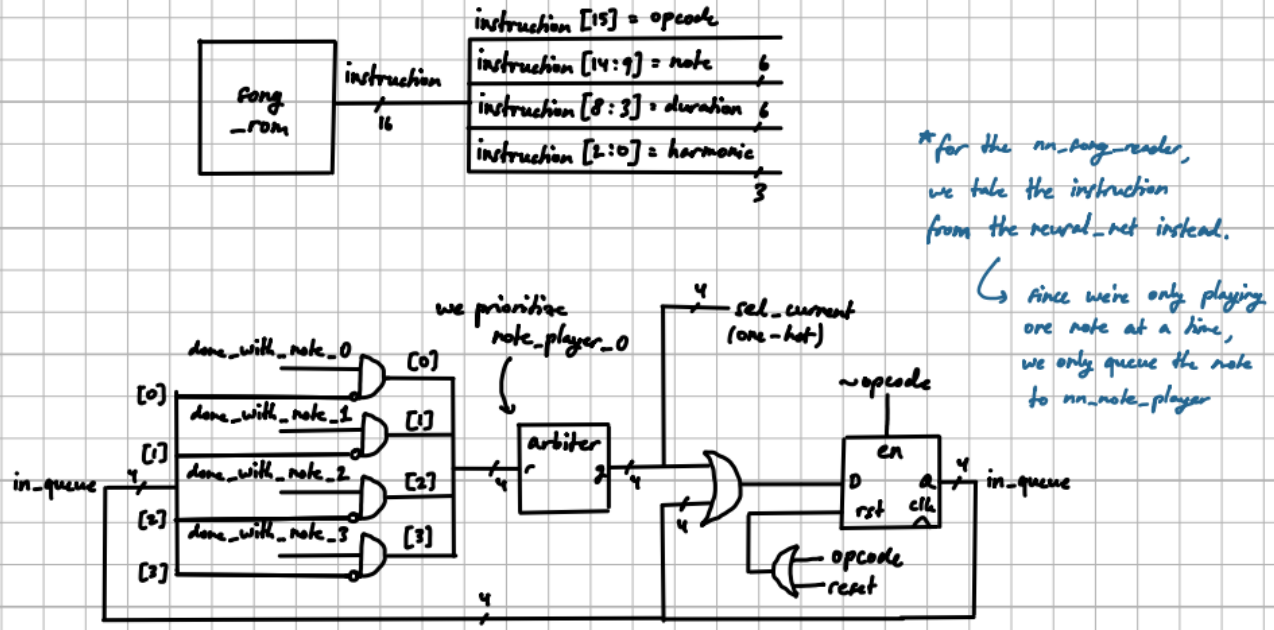


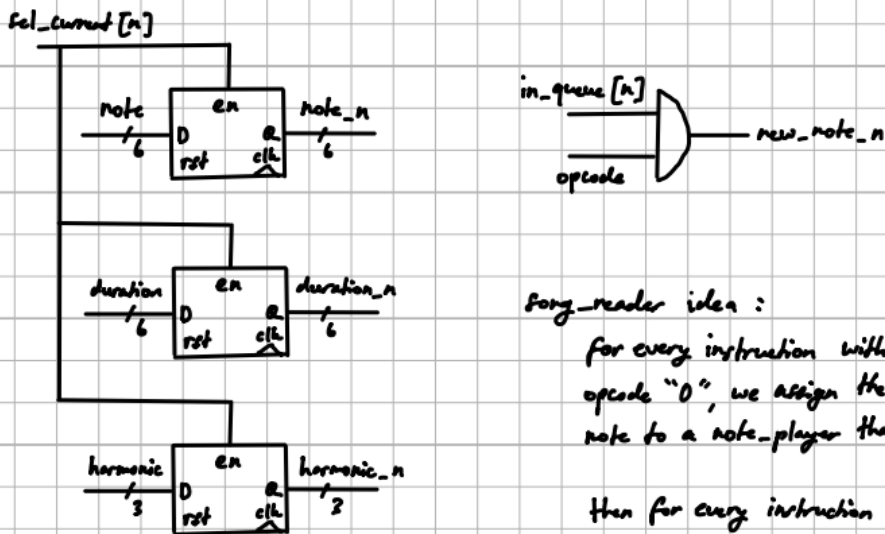
Figure 1: Block diagram for music-player module

song_reader assigns signals to each note_player based on a modified song_rom which contains information about what type of note it is, its duration, and its type of harmonic.

Song_reader implementation



for every note_player (replace "n" with corresponding note_player number),



song_reader iden :

for every instruction with the opcode "0", we assign the incoming note to a note_player that is free.

then for every instruction with the opcode "1", we "flush" the new note by pulsing the signal high.

Figure 2: Block diagram for song_reader prioritizing mechanism

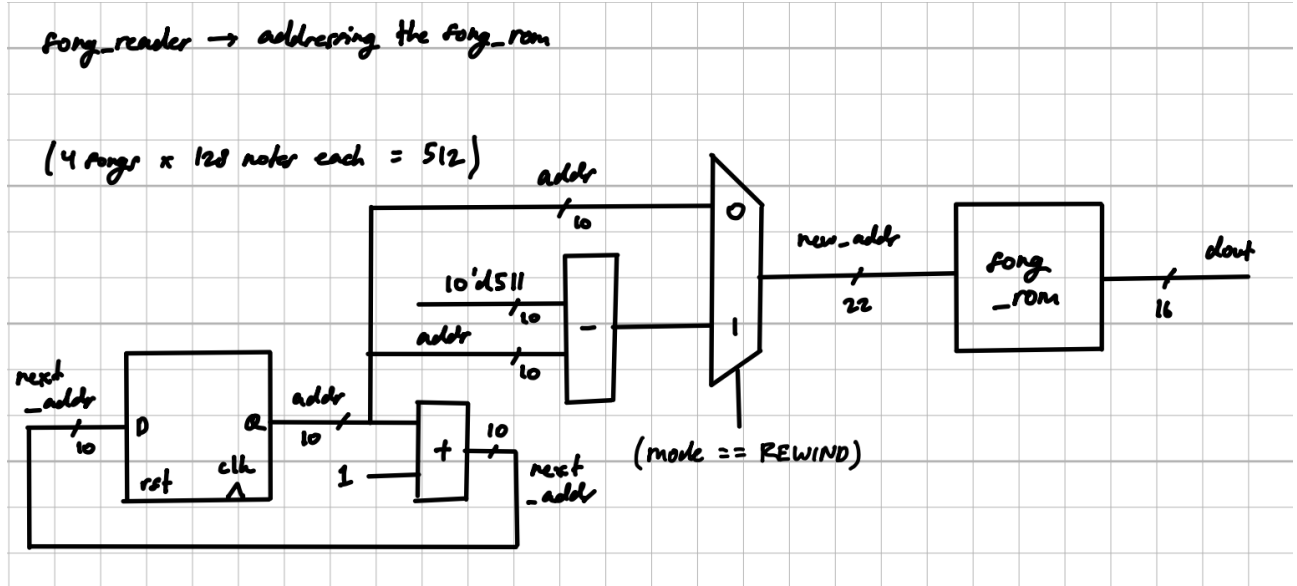


Figure 3: Block diagram for song_rom addressing mechanism in song_reader

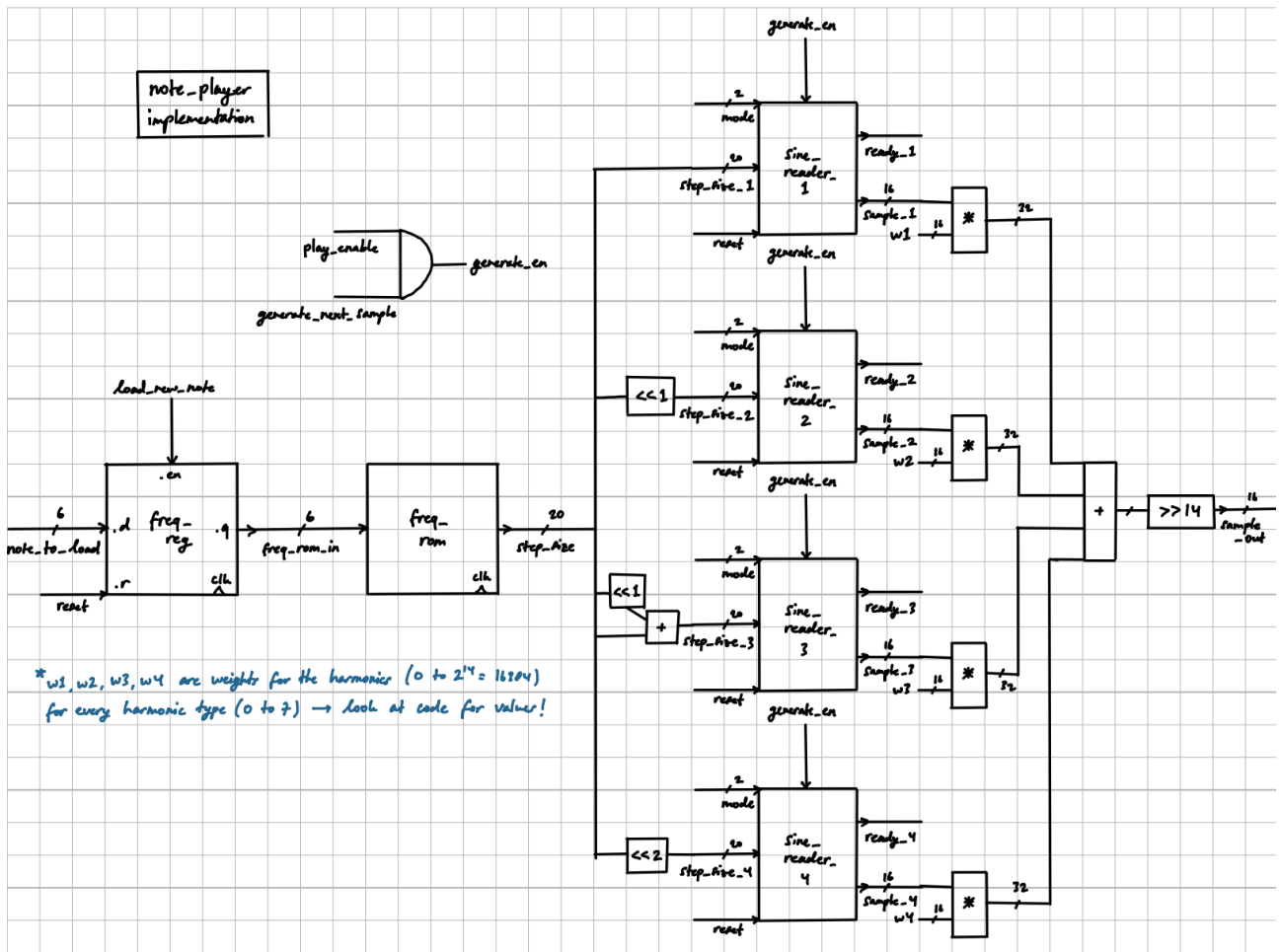
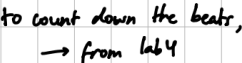


Figure 4: Block diagram for overall implementation of the note_player module



6

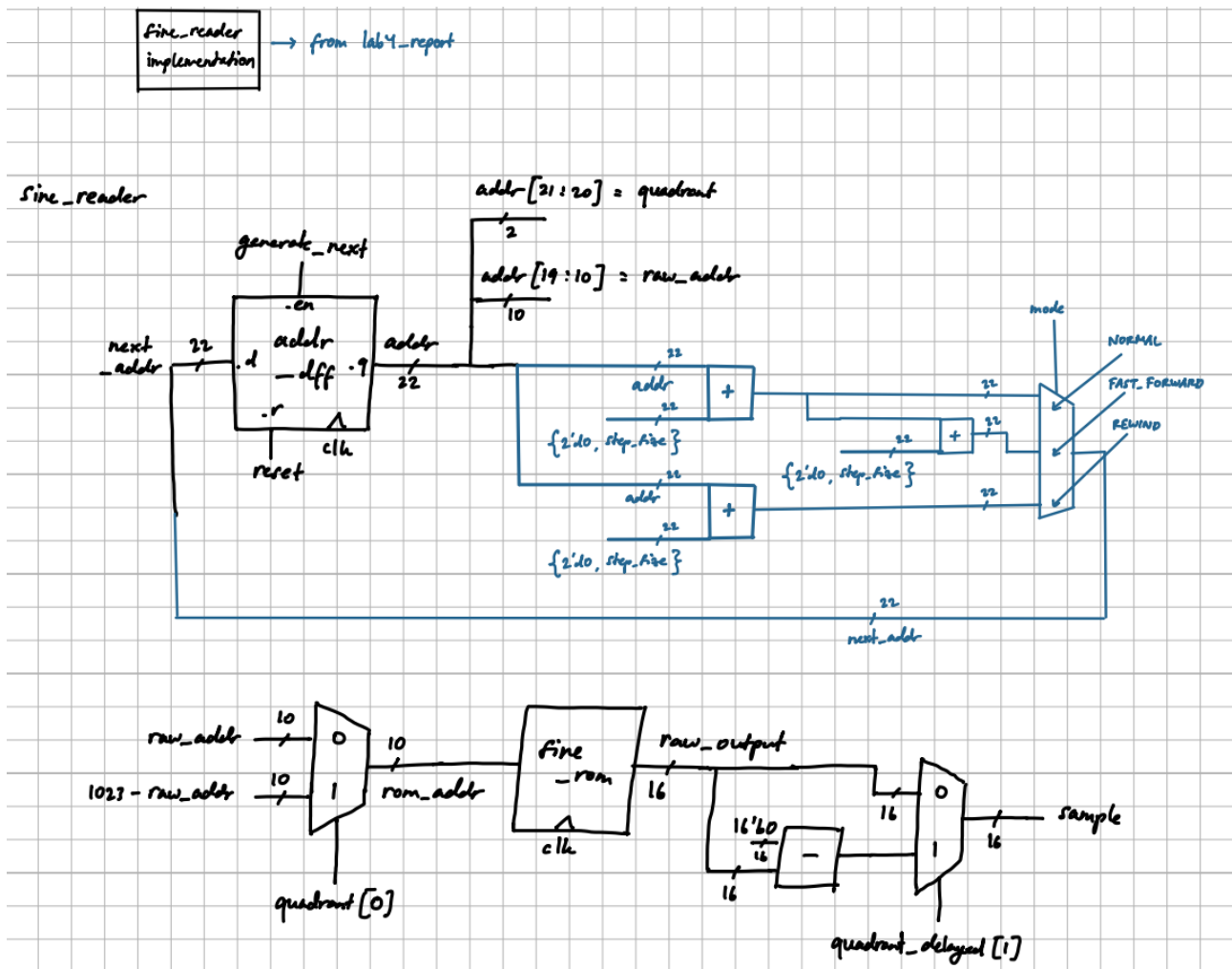
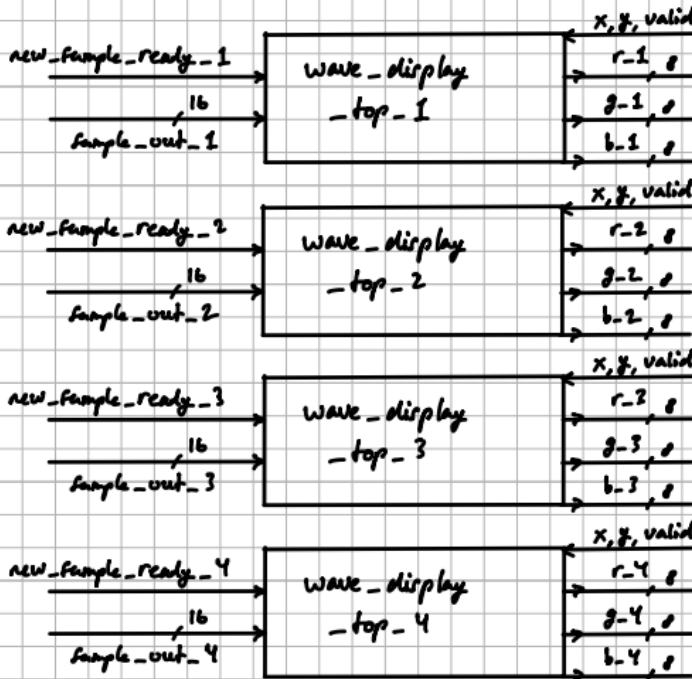


Figure 6: Block diagram for overall implementation of the sine_reader module

Enhanced Waveform Display: each of the 4 notes will be in a different color, with the combined waveform in white.

for each of the 5 total waveforms, we need to have a wave_display_top (lab5) module.



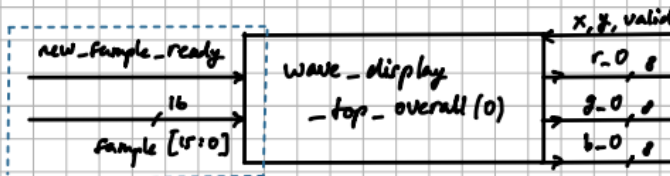
we assign `r-n`, `g-n`, `b-n` to a set value that is distinct from one another.

1 → blue
2 → red
3 → green
4 → pink
overall → white

for every single color signal (`r`, `g`, `b`), we need to prioritize some signal to be put "on top" of others

↳ we prioritize 0 (overall) → 1 → 2
→ 3 → 4

we can use a priority mux for this
↳ arbiter



* when mode == 'GENERATE', the signals here are from `mn_note_player`

for each `wave_display_top`, (replace `n`)

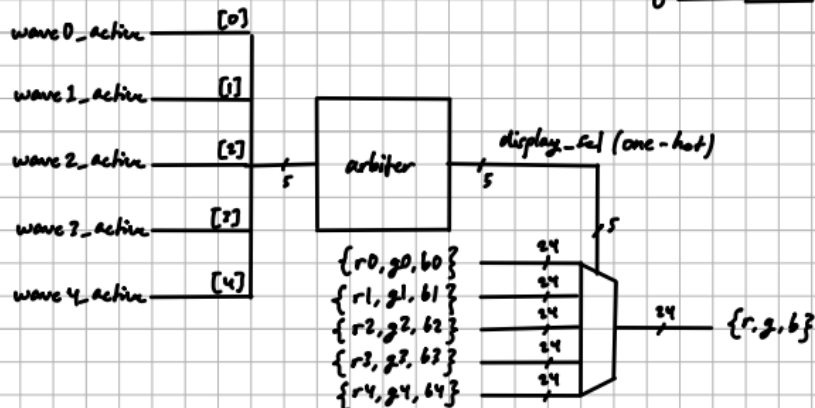
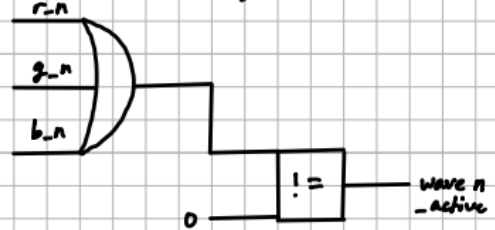


Figure 7: Block diagram for implementation of the Enhanced Waveform Display

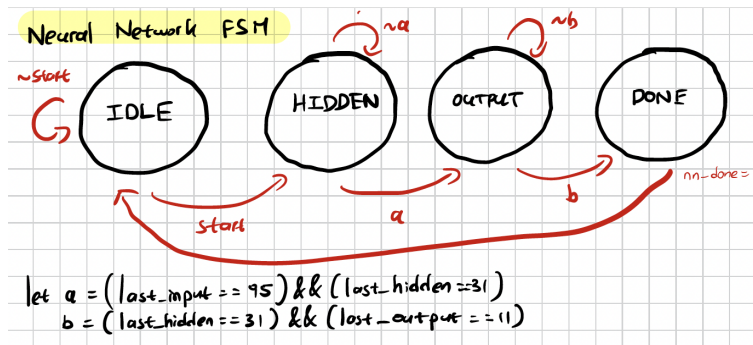


Figure 8: FSM for neural_net module

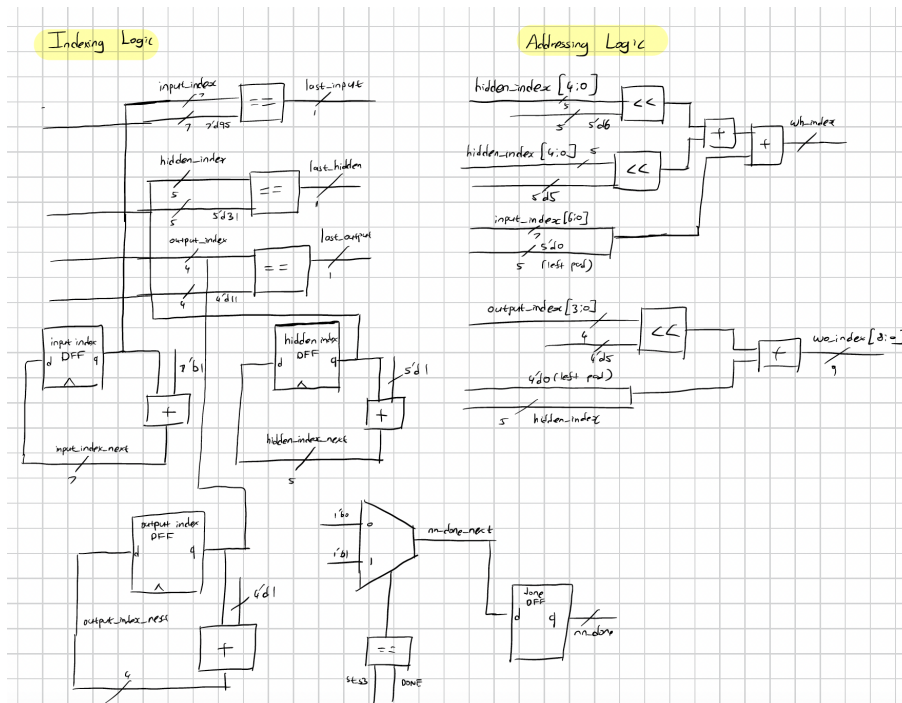


Figure 9: neural_net module block diagram 1

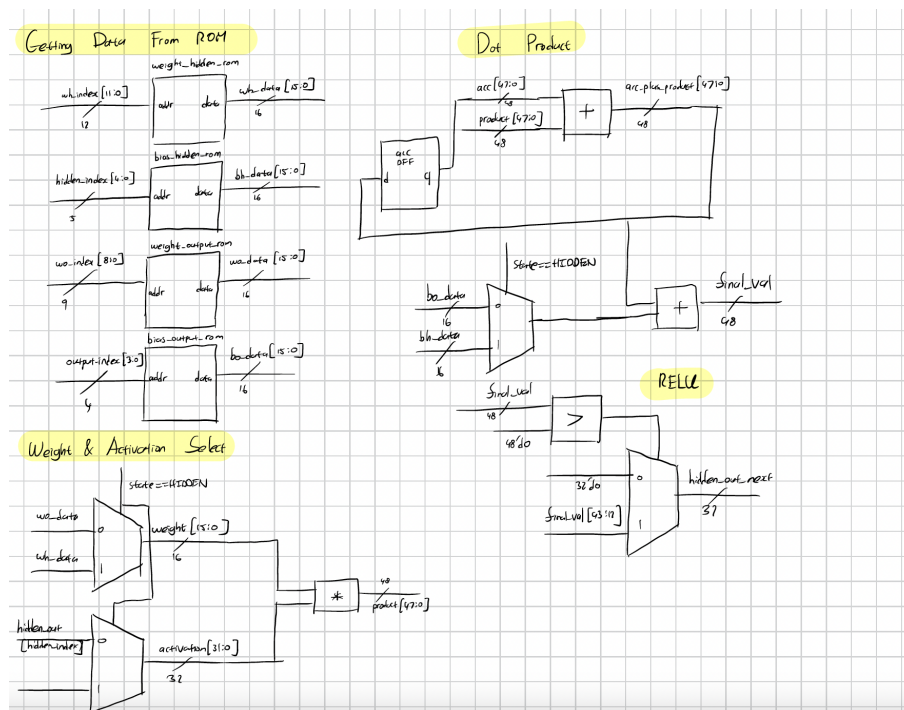


Figure 10: neural_net module block diagram 2